

第 5 讲

CUDA Memory Model

目录

1 课程安排与核心内容	1
1.1 学习目标与技术范围	1
1.2 实验、交流与课程项目	1
2 从处理器执行到 CUDA 存储层次	2
2.1 数据为什么必须先进入寄存器	2
2.2 线程、线程块与设备的可见范围	4
2.3 物理层次、缓存与访问代价	5
2.4 声明限定符与变量生命周期	7
3 基本矩阵乘法：从公式到线程映射	8
3.1 一个输出元素对应一个点积	8
3.2 把输出矩阵划分给二维线程块	10
3.3 线程坐标如何确定输入与输出地址	11
3.4 kernel、启动配置与边界保护	12
4 访存量、算术强度与 Roofline 性能分析	14
4.1 基本矩阵乘法为什么容易受带宽限制	14
4.2 从两个峰值建立 Roofline 模型	15
4.3 向量加法与矩阵乘法的对照	17
5 例题解析与知识归纳	19
5.1 三道例题的条件与推导	19
5.2 关键公式与术语	21

1 课程安排与核心内容

1.1 学习目标与技术范围

CUDA 存储模型连接了两类问题：线程能够访问哪些数据，以及这些访问如何影响计算速度。基本矩阵乘法提供了一个完整例子：为输出元素分配线程，建立正确的地址映射，再根据浮点运算量与访存量分析性能。

学习目标为：了解 CUDA 线程可访问的各类存储器的基本特点；为 Lab 2 的基本矩阵乘法实现做准备；学习评估全局内存访问对性能的影响；了解 GPU 性能指标。对应的技术内容依次为存储层次、基本矩阵乘法，以及算术强度和 Roofline 性能模型。

1.2 实验、交流与课程项目

1.2.1 实验与支持渠道

以下安排对应 2026 年 9 月 8 日的课程信息。Lab 1 的截止时间为当周周五，9 月 11 日；需要先完成 Lab 0，Lab 1 不计入最终成绩。

课程问题通过 Campuswire 发布，课程工作人员只回复在 Campuswire 上发布的问题。提问前应先尝试自行查找答案，包括询问课程聊天机器人。答疑时间同时包含线下与线上形式，需要查看相应时间表。课后事项为阅读教材第 3 章、取得 Delta 账户、设置 GitHub，以及提交 Labs 0 和 1。

1.2.2 课程项目与暂定里程碑

两个项目方向都只实现推理的前向计算 (*inference / forward pass*)，区别在于实现对象与组织形式。

方向	实现内容	组织形式
CNN	LeNet-5 卷积神经网络中的卷积层	个人项目
GPT-2	GPT-2 模型中的各层	团队项目，每组 3-4 人

暂定时间为：Milestone 1 在第 7-8 周，Milestone 2 在第 10-11 周，Milestone 3 在第 15-16 周。以上里程碑为暂定周次。

2 从处理器执行到 CUDA 存储层次

2.1 数据为什么必须先进入寄存器

2.1.1 冯·诺依曼模型中的计算与存储

冯·诺依曼模型 (Von Neumann model) 将计算机划分为处理单元、存储器、输入输出子系统和控制单元。处理单元 (processing unit, PU) 中的算术逻辑单元 (arithmetic logic unit, ALU) 执行算术与逻辑操作，寄存器堆 (register file) 保存正在参与运算的数据。存储器同时保存程序指令和数据；输入输出子系统 (I/O subsystem) 负责与外部环境通信；控制单元 (control unit, CU) 协调指令执行。

在这一执行模型下，运算的操作数位于寄存器中。存储器里的数据需要先装入寄存器，指令也需要从存储器取出并装入指令寄存器 (instruction register, IR)。因此，一条高级语言表达式中的“取值”和“计算”，在执行层面对应不同的工作。

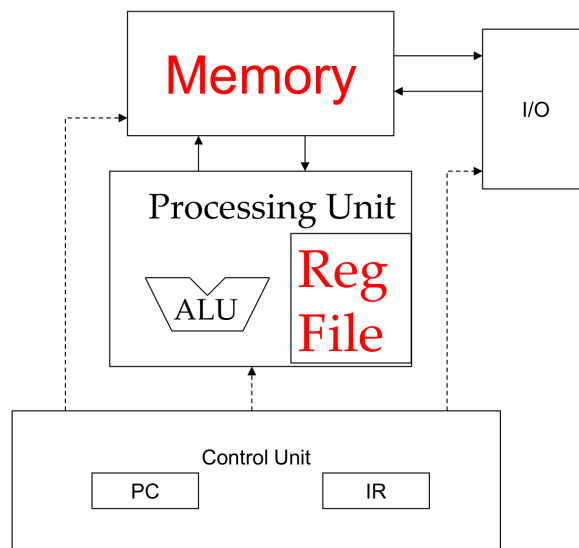


图 1 处理单元通过寄存器保存工作数据，控制单元协调存储、运算与输入输出。图中的 PC 为程序计数器，IR 为指令寄存器。

2.1.2 ADD 与 LOAD 的区别

指令处理可分为取指(*fetch*)、译码(*decode*)、执行(*execute*)和访存(*memory*)几个阶段。三类指令分别承担运算、数据传输与控制流功能。下面两条 LC-3 指令用于说明寄存器运算与数据装载的差别。

```
ADD R1, R2, R3
LDR R4, R6, #3
```

ADD R1, R2, R3 读取 R2 和 R3，进行加法，再把结果写入 R1。两个输入都已在寄存器中，其数据运算不需要再从数据存储器读取操作数。用寄存器内容 R_i 表示，可写为

$$R_1 \leftarrow R_2 + R_3.$$

LDR R4, R6, ##3 先读取 R6，将其内容加上偏移量 3，形成有效地址，再从该地址读出数据并写入 R4。若 $\text{Mem}[a]$ 表示地址 a 处的内容，则

$$a \leftarrow R_6 + 3, \quad R_4 \leftarrow \text{Mem}[a].$$

两条指令本身都需要被取出和译码；差别在于 LOAD 还显式访问其计算出的数据地址。这里的 3 保留 LC-3 示例中的地址偏移含义，不将其转换为 CUDA 数组的字节步长。

2.1.3 延迟与容量的差别

寄存器接近运算单元，适合保存当前工作值；主存容量大，适合保存完整数据集。简化的数量级对照为：寄存器访问约 1 个周期，内存访问约数百个周期；CPU 的寄存器数量为数百量级，GPU 的每个流式多处理器(*streaming multiprocessor, SM*)拥有 $O(10^4)$ 量级的寄存器，而内存容量可达到 GB 或更大。

从计算式看数据流

对于“从数组读取两个数并相乘累加”，数据先被读入可供运算使用的寄存器，再进行乘法和累加。反复使用已经取到的工作值，可以减少再次读取远端存储器的需求。CUDA 的不同存储空间，为线程私有工作值、块内协作和大规模数据集提供了不同的位置。

2.2 线程、线程块与设备的可见范围

存储空间可以从两个角度理解：**作用域(scope)**决定哪些线程能够访问同一份数据；**物理位置(physical location)**决定数据处于 SM 内部、设备内存，还是相应的缓存路径。变量的**生命周期(lifetime)**则表示这份存储随线程、线程块或应用程序存在的时间范围。

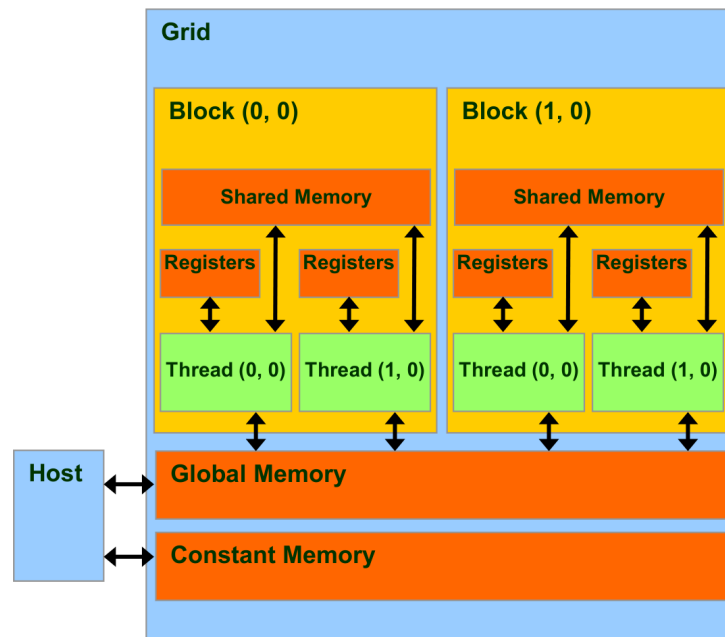


图 2 线程持有自己的寄存器，同一线程块的线程共同访问该块的共享内存；全局内存与常量内存对网格中的线程可见。

2.2.1 线程私有：寄存器与局部内存

寄存器(registers)保存单个线程的私有工作值，访问延迟最低。例如，矩阵乘法中的累加器只需要由当前线程更新，适合用一个自动标量变量表示，其典型位置就是寄存器。

局部内存(local memory)同样属于单个线程的私有地址空间，但物理上由设备内存支持，并通常经过 L1/L2 缓存。较大的自动对象，以及**寄存器溢出(register spill)**所需的存储，都可能使用这一空间。这里的 **local** 描述的是线程私有的作用域，物理位置仍然可能远离运算单元。

这一区分直接影响对代码的理解：两个变量都只在一个线程中使用，并不代表它们具有相同的访问代价。自动标量通常使用寄存器；线程私有数组可能使用寄存器，也可能使用局部内存，具体分配由编译器处理。

2.2.2 块内共享：共享内存

共享内存 (*shared memory*) 位于 **SM** 上，由同一**线程块** (*thread block*) 中的线程共同访问，生命周期与线程块对应。不同线程块各自拥有独立的共享内存存储。

共享内存适合承担块内协作和数据复用：某个线程把数据写入本块的共享内存后，其他需要该数据的块内线程可以从这份共享存储中读取。因而，当多个线程需要反复使用相同输入时，共享内存提供了显式组织复用的位置。共享内存分块矩阵乘法利用这一机制组织输入复用。

2.2.3 设备可见：全局内存与常量内存

全局内存 (*global memory*) 对设备上的线程可见，适合保存主数据集，例如输入矩阵与输出矩阵。它支持线程读取和写入，容量大，但访问延迟高。代码中的全局地址访问也可能由缓存提供数据，因此其请求与最终到达设备内存的流量需要分开计数。

常量内存 (*constant memory*) 保存设备线程只读的数据，并通过**常量缓存** (*constant cache*) 加速访问。**广播或一致访问** (*broadcast / uniform access*)，即多个线程读取相同的数据，是它特别适合的访问情形。全局内存和常量内存的可见范围都可以覆盖一个网格；对设备变量而言，其生命周期在声明表中列为应用程序级。

2.3 物理层次、缓存与访问代价

2.3.1 从 SM 到设备内存

在硬件视图中，每个 **SM** 内部具有寄存器、共享内存、**L1 缓存** 与 **常量缓存**；多个 **SM** 通过下一级 **L2 缓存** 访问全局内存。**缓存** (*cache*) 保存已访问数据的副本，使后续访问有机会在更接近计算单元的位置得到响应。局部内存与全局内存的访问流量都可以经过缓存。

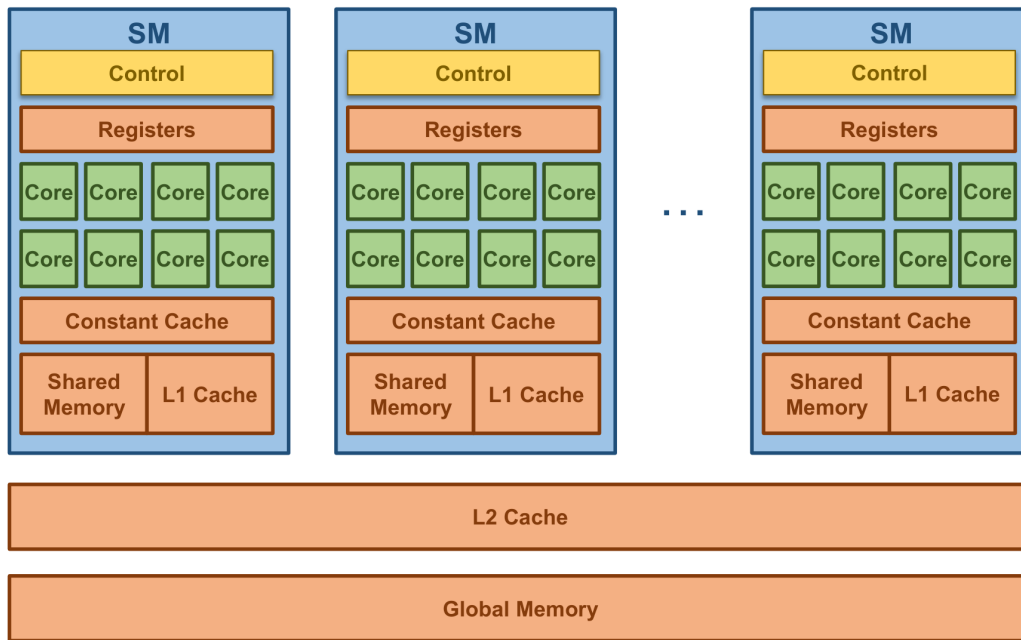


图 3 寄存器与块内共享存储位于 SM 内部，L2 缓存位于多个 SM 与全局内存之间。寄存器、共享内存与缓存分别承担不同的数据保存与复用任务。

表 1 CUDA 存储层次的作用域、位置与管理方式。

类型	作用域	典型位置	管理方式	主要用途
寄存器	线程	SM	编译器	私有工作值
局部内存	线程	设备内存及缓存	编译器	溢出、较大自动对象
共享内存	块	SM	程序员	块内协作与复用
全局内存	设备/网格	HBM / DRAM	程序员	主数据集
常量内存	设备/网格	DRAM 与常量缓存	程序员	一致的只读数据
L1 / L2	缓存层	片上	主要由硬件	缓存全局/局部访问

这里“程序员管理”指需要在程序中安排相应存储及数据使用；寄存器与局部内存之间的自动变量分配则主要由编译器处理。

2.3.2 周期数的使用范围

简化的程序员视图给出寄存器约 1 周期、共享内存约 5 周期、全局内存约 500 周期，以及命中缓存的常量内存约 5 周期。硬件层次图另将共享内存和常量缓存标为数十周期、全局内存标为数百周期。

这些示意值用于建立层次之间的代价差别：寄存器访问最快，片上共享存储与缓存提供较低延迟，设备内存访问具有更高延迟。两个图的数值口径分别保留，不能从中得到某一指定 GPU 上各类访问的固定延迟。

2.4 声明限定符与变量生命周期

2.4.1 从声明读出存储空间

CUDA 类型限定符 (*type qualifiers*) 让变量声明表达其存储属性。下表中的设备变量声明对应设备侧存储；函数内没有限定符的自动变量则按线程独立产生。

表 2 变量声明、典型存储空间与生命周期。

声明形式	存储空间	作用域	生命周期
<code>__device__ int globalVar;</code>	全局	设备/网格	应用程序
<code>__device__ __constant__ int constantVar;</code>	常量	设备/网格	应用程序
<code>__device__ __shared__ int sharedVar;</code>	共享	块	块
<code>int localVar;</code>	通常为寄存器	线程	线程
<code>int localArr[N];</code>	寄存器或局部内存	线程	线程

声明中若已有 `__shared__` 或 `__constant__`，可以省略 `__device__`。

单独使用 `__device__` 的变量声明不能放在函数内部。表中的 N 表示数组长度。

2.4.2 自动变量与线程私有状态

设一个 kernel 中声明了 `int Row;` 和 `float Pvalue;`。每个线程都有自己的行索引和累加器：线程 A 更新其 `Pvalue`，不会因此更新线程 B 的 `Pvalue`。这正是用一个线程独立计算一个输出元素所需的私有状态。

当一个线程块中声明了共享变量，该块内的线程共同使用这份存储；另一个块中的同名共享变量属于另一个独立实例。因而，变量名相同与物理上访问同一份数据之间的关系，取决于变量所属的存储空间和作用域。

判断变量的三个问题

先确定谁需要访问它，再确定它的典型物理位置，最后检查它需要存在多久。线程私有工作值、块内共享中间结果、设备可见的大型输入输出，在这三个问题上分别对应不同的选择。

3 基本矩阵乘法：从公式到线程映射

3.1 一个输出元素对应一个点积

3.1.1 矩阵、下标与行主序地址

设输入矩阵 M, N 和输出矩阵 P 均为 $W \times W$ ，其中 W 对应代码中的 **Width**。采用与 C/CUDA 代码一致的零起始下标，行、列和求和下标分别为 i, j, k ，取值范围均为 $0, \dots, W-1$ 。矩阵乘法定义为

$$P_{ij} = \sum_{k=0}^{W-1} M_{ik} N_{kj}. \quad (1)$$

一个 P_{ij} 取决于 M 的第 i 行和 N 的第 j 列：将两者逐项相乘，再把这些乘积相加。数学公式与数组寻址均使用零起始下标。

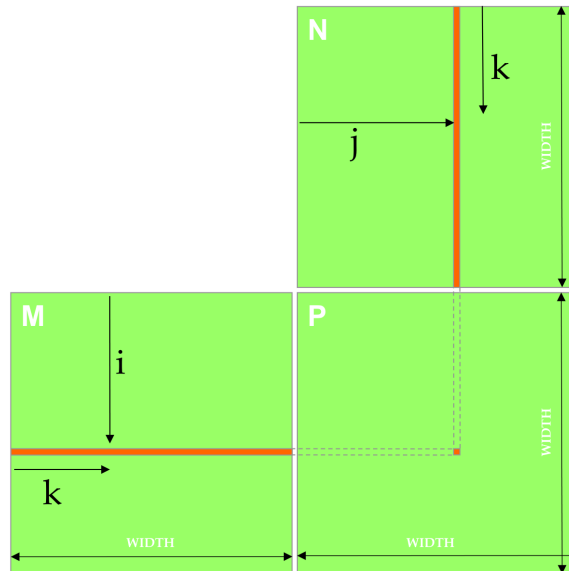


图 4 沿 M 的一行和 N 的一列取数，计算它们在 P 中交会位置的元素。

行主序 (*row-major order*) 将同一行的元素连续存储。在每行包含 W 个元素时，跳过前 i 行需要跳过 iW 个元素，再在当前行向右移动 j 个元素，因此二维位置与一维偏移之间的对应关系为

$$(i, j) \longleftrightarrow iW + j. \quad (2)$$

对公式(1)中的三个矩阵，这给出

$$M_{ik} \leftrightarrow M[i * \text{Width} + k], \quad N_{kj} \leftrightarrow N[k * \text{Width} + j],$$

$$P_{ij} \leftrightarrow P[i * \text{Width} + j].$$

这些表达式的下标以元素为单位；数组元素为 `float` 时，访存字节量另按每个元素 4 B 计算。

3.1.2 CPU 三重循环

固定 i 与 j 后，内层循环用一个私有累加器计算点积。外层的两重循环则遍历所有输出位置。

代码 1 基本 CPU 矩阵乘法，保留行主序寻址和三重循环结构。

```

1 void MatrixMul(float *M, float *N, float *P, int Width)
2 {
3     for (int i = 0; i < Width; ++i) {
4         for (int j = 0; j < Width; ++j) {
5             float sum = 0.0f;
6             for (int k = 0; k < Width; ++k) {
7                 float a = M[i * Width + k];
8                 float b = N[k * Width + j];
9                 sum += a * b;
10            }
11            P[i * Width + j] = sum;
12        }
13    }
14 }
```

3.1.3 并行化输出元素

不同的 P_{ij} 使用不同的累加器，并写入不同的输出位置；它们共同读取 M, N ，但计算一个输出元素不依赖另一个输出元素的完成。因此，可以将外层的 (i, j) 遍历映射为并行线程，让一个线程负责一个输出元素，而该线程内部仍顺序执行 k 循环。

把内层求和本身并行化会涉及归约 (*reduction*)。浮点加法通常不满足结合律，改变求和顺序可能改变舍入结果；每个线程内部的顺序累加保持固定的求和次序。

CPU 循环与 GPU 线程的对应关系

CPU 的两个外层循环负责“选择输出位置”，GPU 用线程的二维索引完成这项选择。内层循环负责“计算该位置上的完整点积”，仍由被分配到该位置的线程执行。

3.2 把输出矩阵划分给二维线程块

3.2.1 线程块、输出分块与网格大小

令 T 为 `TILE_WIDTH`，即一个方形线程块在两个方向上的宽度。一个 $T \times T$ 的线程块包含 T^2 个线程，负责输出矩阵中的一个 $T \times T$ 子区域，称为一个分块 (tile)。

当 W 能被 T 整除时，两个方向各需要 W/T 个块。一般情况下需要向上取整，使边缘位置也被覆盖：

$$G_x = G_y = \left\lceil \frac{W}{T} \right\rceil, \quad N_{\text{blocks}} = G_x G_y = \left\lceil \frac{W}{T} \right\rceil^2. \quad (3)$$

这里的分块首先用于分配输出计算。共享内存分块矩阵乘法进一步把 M, N 的输入子矩阵显式装入共享内存，以组织输入数据复用。

3.2.2 三个网格配置例子

矩阵宽 W	块宽 T	每块线程数	网格尺寸	总块数
4	2	$2^2 = 4$	2×2	4
8	2	$2^2 = 4$	4×4	16
8	4	$4^2 = 16$	2×2	4

表 3 输出分块大小决定单块工作量及网格中线程块的数量。

P _{0,0}	P _{0,1}	P _{0,2}	P _{0,3}	P _{0,4}	P _{0,5}	P _{0,6}	P _{0,7}
P _{1,0}	P _{1,1}	P _{1,2}	P _{1,3}	P _{1,4}	P _{1,5}	P _{1,6}	P _{1,7}
P _{2,0}	P _{2,1}	P _{2,2}	P _{2,3}	P _{2,4}	P _{2,5}	P _{2,6}	P _{2,7}
P _{3,0}	P _{3,1}	P _{3,2}	P _{3,3}	P _{3,4}	P _{3,5}	P _{3,6}	P _{3,7}
P _{4,0}	P _{4,1}	P _{4,2}	P _{4,3}	P _{4,4}	P _{4,5}	P _{4,6}	P _{4,7}
P _{5,0}	P _{5,1}	P _{5,2}	P _{5,3}	P _{5,4}	P _{5,5}	P _{5,6}	P _{5,7}
P _{6,0}	P _{6,1}	P _{6,2}	P _{6,3}	P _{6,4}	P _{6,5}	P _{6,6}	P _{6,7}
P _{7,0}	P _{7,1}	P _{7,2}	P _{7,3}	P _{7,4}	P _{7,5}	P _{7,6}	P _{7,7}

(a) $W = 8, T = 2$: 16 个块。

P _{0,0}	P _{0,1}	P _{0,2}	P _{0,3}	P _{0,4}	P _{0,5}	P _{0,6}	P _{0,7}
P _{1,0}	P _{1,1}	P _{1,2}	P _{1,3}	P _{1,4}	P _{1,5}	P _{1,6}	P _{1,7}
P _{2,0}	P _{2,1}	P _{2,2}	P _{2,3}	P _{2,4}	P _{2,5}	P _{2,6}	P _{2,7}
P _{3,0}	P _{3,1}	P _{3,2}	P _{3,3}	P _{3,4}	P _{3,5}	P _{3,6}	P _{3,7}
P _{4,0}	P _{4,1}	P _{4,2}	P _{4,3}	P _{4,4}	P _{4,5}	P _{4,6}	P _{4,7}
P _{5,0}	P _{5,1}	P _{5,2}	P _{5,3}	P _{5,4}	P _{5,5}	P _{5,6}	P _{5,7}
P _{6,0}	P _{6,1}	P _{6,2}	P _{6,3}	P _{6,4}	P _{6,5}	P _{6,6}	P _{6,7}
P _{7,0}	P _{7,1}	P _{7,2}	P _{7,3}	P _{7,4}	P _{7,5}	P _{7,6}	P _{7,7}

(b) $W = 8, T = 4$: 4 个块。

图 5 相同的 8×8 输出矩阵可以使用不同大小的输出分块；粗线表示块之间的边界。

对这两个 $W = 8$ 的整除例子，总线程数都是 64，每个线程仍计算一个输出元素。改变 T 调整了线程分组，但基本 kernel 的每个有效线程仍然执行长度为 W 的点积。

3.3 线程坐标如何确定输入与输出地址

3.3.1 从块索引与块内索引得到行列

记块索引为 (b_x, b_y) ，块内线程索引为 (t_x, t_y) ，块尺寸为 (B_x, B_y) 。它们分别对应 `blockIdx`、`threadIdx` 与 `blockDim` 的两个分量。线程负责的全局行、列为

$$\text{Row} = b_y B_y + t_y, \quad \text{Col} = b_x B_x + t_x. \quad (4)$$

x 方向沿矩阵的列变化， y 方向沿行变化。乘以块尺寸给出当前块的起点，再加上块内偏移，得到整个输出矩阵中的位置。

当 $T = 2$ 且 $(b_x, b_y) = (0, 0)$ 时，该块负责行 0,1 和列 0,1，四个线程与输出的对应关系如下。

(t_x, t_y)	Row	Col	负责的输出
(0,0)	0	0	$P_{0,0}$
(1,0)	0	1	$P_{0,1}$
(0,1)	1	0	$P_{1,0}$
(1,1)	1	1	$P_{1,1}$

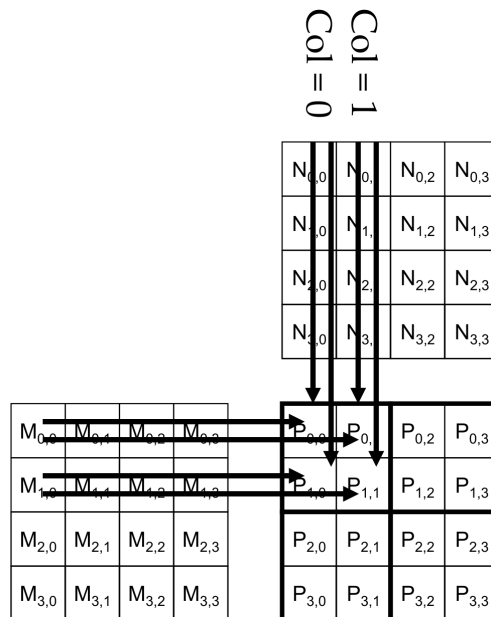


图 6 左上角线程块中的四个线程读取 M 的前两行与 N 的前两列，各自形成一个完整点积。

向右相邻的块满足 $(b_x, b_y) = (1, 0)$ ，其行仍为 0, 1，列变为 2, 3。CUDA 块坐标按 (b_x, b_y) 排列， b_x 表示水平方向， b_y 表示垂直方向。

3.3.2 固定一个线程，追踪整个点积

取 $W = 4$ 、 $\text{Row} = 1$ 、 $\text{Col} = 2$ 。线程最终写入 $P_{1,2}$ ，对应一维位置 $1 \times 4 + 2 = 6$ 。内层循环的四次访问为

k	M 的元素与偏移	N 的元素与偏移	对累加器的贡献
0	$M_{1,0}$ ，偏移 4	$N_{0,2}$ ，偏移 2	$M_{1,0}N_{0,2}$
1	$M_{1,1}$ ，偏移 5	$N_{1,2}$ ，偏移 6	$M_{1,1}N_{1,2}$
2	$M_{1,2}$ ，偏移 6	$N_{2,2}$ ，偏移 10	$M_{1,2}N_{2,2}$
3	$M_{1,3}$ ，偏移 7	$N_{3,2}$ ，偏移 14	$M_{1,3}N_{3,2}$

沿 k 增长时， M 的地址每次增加 1， N 的地址每次增加 W 。这是一个线程内部沿一行和一系列取数所产生的地址序列。每次读入两个元素并相乘累加，四次循环结束后，只需将累加器写入一次输出数组。

3.4 kernel、启动配置与边界保护

3.4.1 完整的基本 kernel

代码 2 一个线程计算一个输出元素的基本矩阵乘法 kernel。

```

1  __global__
2  void MatrixMulKernel(float *d_M, float *d_N,
3                      float *d_P, int Width)
4  {
5      int Row = blockIdx.y * blockDim.y + threadIdx.y;
6      int Col = blockIdx.x * blockDim.x + threadIdx.x;
7
8      if ((Row < Width) && (Col < Width)) {
9          float Pvalue = 0.0f;
10         for (int k = 0; k < Width; ++k) {
11             Pvalue += d_M[Row * Width + k]
12                     * d_N[k * Width + Col];
13         }
14         d_P[Row * Width + Col] = Pvalue;
15     }
16 }
```

Row 、 Col 、 k 与 Pvalue 均为线程自己的状态。 d_M 、 d_N 和 d_P 指向设备数组，其中循环内两处下标访问读取全局数据，循环后的下标访问写回输出。 Pvalue 在整个循环中保

留累加结果，避免每做一次乘加就把部分和写回输出数组。

3.4.2 主机侧启动配置

令 `TILE_WIDTH` 为预先定义的正整数常量，三个设备数组已经准备好，`Width` 为矩阵宽度。启动片段如下；`ceil` 对应向上取整，C++ 中可在包含 `<cmath>` 后使用 `std::ceil`。

代码 3 二维网格与二维线程块的启动片段。

```
1 dim3 dimGrid(std::ceil((1.0 * Width) / TILE_WIDTH),
2             std::ceil((1.0 * Width) / TILE_WIDTH), 1);
3 dim3 dimBlock(TILE_WIDTH, TILE_WIDTH, 1);
4
5 MatrixMulKernel<<<dimGrid, dimBlock>>>(
6     d_M, d_N, d_P, Width);
```

$1.0 * Width$ 使除法在浮点数范围内进行，从而在向上取整前保留余数对应的小数部分。对正整数 W, T ，同一数量也可用整数算术表达为

$$\left\lceil \frac{W}{T} \right\rceil = \left\lfloor \frac{W + T - 1}{T} \right\rfloor,$$

因此可以用 $(Width + TILE_WIDTH - 1) / TILE_WIDTH$ 计算每个方向的块数。两个表达式都表示“有余数时再增加一个块”。

3.4.3 向上取整后为什么需要边界判断

取 $W = 5, T = 2$ ，每个方向需要 $\lceil 5/2 \rceil = 3$ 个块，共 9 个块、36 个线程。它们覆盖一个 6×6 的线程坐标区域，而有效输出只包含 $5 \times 5 = 25$ 个位置。因此，有 11 个线程对应超出矩阵边缘的坐标。

`if ((Row < Width) && (Col < Width))` 同时检查行与列，使这些边缘线程跳过整个点积和输出写入。有效线程内部有 $0 \leq k < W$ ，因此 M 的列索引与 N 的行索引也位于范围内。

任一有效位置 (r, c) 都可以唯一分解为

$$b_y = \lfloor r/T \rfloor, \quad t_y = r \bmod T, \quad b_x = \lfloor c/T \rfloor, \quad t_x = c \bmod T.$$

这说明向上取整的网格覆盖了所有有效输出，并且每个输出都被唯一一个线程负责。不同输出可以重复读取相同输入，但写入目标彼此不同。

4 访存量、算术强度与 Roofline 性能分析

4.1 基本矩阵乘法为什么容易受带宽限制

4.1.1 逐次迭代统计操作数与运算量

基本矩阵乘法将输出位置分配给大量线程，但每个线程在每次 k 循环中都要读取 $d_M[\text{Row} * \text{Width} + k]$ 和 $d_N[k * \text{Width} + \text{Col}]$ 。两个 `float` 元素各为 4 B，因此每次迭代请求 8 B 输入数据，同时执行一次浮点乘法 and 一次浮点加法。

浮点运算 (*floating-point operation, FLOP*) 是这里的运算计数单位：一次乘法与一次加法合计 2 FLOP。将数据需求除以运算量，得到

$$R_{\text{req}} = \frac{8 \text{ B}}{2 \text{ FLOP}} = 4 \text{ B/FLOP}. \quad (5)$$

其倒数是每读取一字节所完成的运算量，即**算术强度** (*arithmetic intensity*)。用 I 表示，基本 kernel 在不计缓存复用、忽略最终写回的口径下有

$$I_{\text{req}} = \frac{2}{8} = 0.25 \text{ FLOP/B}. \quad (6)$$

设总浮点运算量为 F ，源代码请求的总字节量为 Q_{req} 。保留最后一次输出写回时，计数可逐层展开如下。

计数范围	浮点运算量	请求的字节量
一个线程的一次 k 迭代	2 FLOP	8 B 输入
一个有效线程的完整点积	$2W$ FLOP	$8W + 4 \text{ B}$
整个 $W \times W$ 输出	$F = 2W^3$ FLOP	$Q_{\text{req}} = 8W^3 + 4W^2 \text{ B}$

由此得到更完整的请求级算术强度

$$I_{\text{req}} = \frac{2W^3}{8W^3 + 4W^2} = \frac{W}{4W + 2} \xrightarrow{W \rightarrow \infty} \frac{1}{4} \text{ FLOP/B}. \quad (7)$$

输出写入在总请求流量中的比例随 W 增大而减小，因此大矩阵分析中忽略它，可以得到简洁的 0.25 FLOP/B 估计。

请求流量与设备内存流量

上述 Q_{req} 统计各线程在代码中请求的操作数。缓存复用可以让多个请求由已缓存数据响应，使实际到达所建模存储层的字节量 Q 减少。用设备内存带宽估计性能时，算术强度应按该层的 $I = F/Q$ 计算；忽略缓存复用时，才把上述请求计数作为该层流量的近似。

4.1.2 150 GB/s 如何得到 37.5 GFLOP/s

考虑给定的示例处理器：计算吞吐上限为 1000 GFLOP/s，全局内存带宽为 150 GB/s。若每执行 1 FLOP 都需要从该层取得 4 B 数据，则内存每秒能支撑的运算量为

$$\frac{150 \text{ GB/s}}{4 \text{ B/FLOP}} = 37.5 \text{ GFLOP/s.} \quad (8)$$

也可以写成 $150 \text{ GB/s} \times 0.25 \text{ FLOP/B}$ 。字节单位消去后，结果为运算吞吐率。

这解释了瓶颈：计算单元每秒最多能够完成 1000 G 次浮点运算，但当前的数据需求让带宽只能支撑其中的 37.5 G 次。大量输出线程提供了并行任务，却没有减少每单位计算需要搬运的数据。

在这一示例中，37.5 GFLOP/s 是按给定访存模型得到的带宽上限；原材料给出的执行表现约为 25 GFLOP/s，并解释为内存并非始终处于满负荷状态。提高性能的方向是减少重复的全局数据访问，使每次搬入的数据参与更多运算。上述数值均属于该教学示例。

4.2 从两个峰值建立 Roofline 模型

4.2.1 运算吞吐率与内存带宽

浮点运算吞吐率 (*FLOPS rate*) 表示每秒能够完成多少浮点运算，单位为 FLOP/s。内存带宽 (*memory bandwidth*) 表示每秒能够提供多少数据，单位为 B/s。前者度量计算供给，后者度量数据供给。

用 C_{peak} 表示峰值计算吞吐率， B_{peak} 表示所建模存储层的峰值带宽；设一个 kernel 的总工作量为 F FLOP，在该层传输 Q B 数据，执行时间为 t 。则算术强度和实际运算吞吐率分别为

$$I = \frac{F}{Q} \quad (\text{FLOP/B}), \quad \mathcal{P} = \frac{F}{t} \quad (\text{FLOP/s}). \quad (9)$$

F 是运算总量， $\mathcal{P}$ 是运算速率；相应地， Q 是数据总量， Q/t 是平均数据传输速率。计算例子中的 G 和 T 按十进制量级 10^9 、 10^{12} 使用。

4.2.2 性能上限与时间下限

计算单元提供了 C_{peak} 的吞吐上限；内存以 B_{peak} B/s 供给数据，每字节可支持 I FLOP，因而给出 $B_{\text{peak}}I$ 的另一条上限。两项限制必须同时满足：

$$\mathcal{P} \leq \min(C_{\text{peak}}, B_{\text{peak}}I). \quad (10)$$

这就是 **Roofline 模型 (roofline model)** 的基本关系。

用执行时间表达同一限制，计算所需时间至少为 F/C_{peak} ，数据传输所需时间至少为 Q/B_{peak} ，因此

$$t \geq t_{\min} = \max\left(\frac{F}{C_{\text{peak}}}, \frac{Q}{B_{\text{peak}}}\right). \quad (11)$$

这里取两项的最大值，表达两条下限必须同时成立；该式没有把两个阶段假定为必然串行。即便达到较小的一项，仍须满足较大的时间需求。

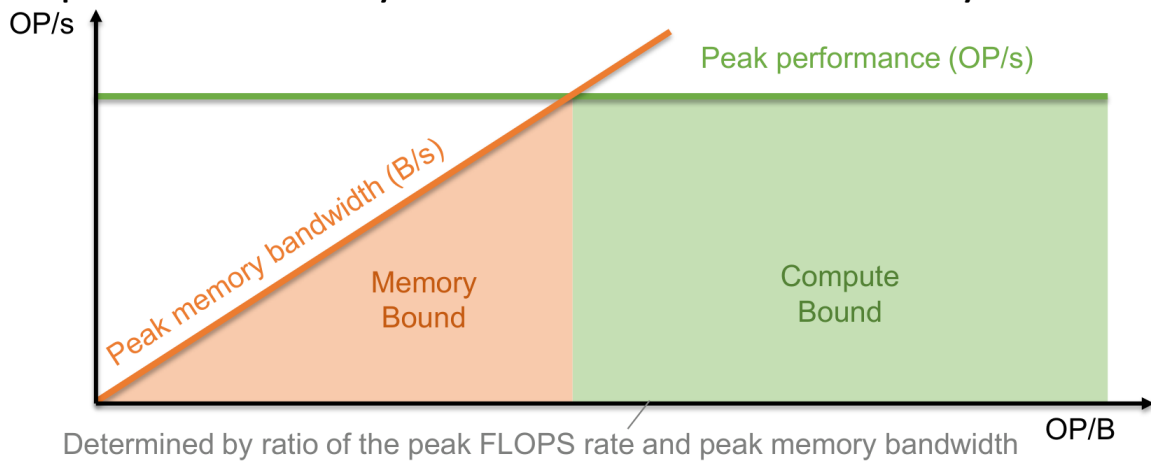


图 7 横轴为算术强度，纵轴为运算吞吐率。斜线给出带宽限制，水平线给出计算限制；可达到的性能位于两条限制的下方。

4.2.3 屋脊点与受限类型

两条上限相交的位置称为**屋脊点 (ridge point)**。令 $C_{\text{peak}} = B_{\text{peak}}I_*$ ，得到

$$I_* = \frac{C_{\text{peak}}}{B_{\text{peak}}}. \quad (12)$$

当 $I < I_*$ 时，带宽提供的性能上限较低，对应**内存受限 (memory-bound)** 区域：计算核心可能等待内存提供数据。当 $I > I_*$ 时，计算吞吐上限较低，对应**计算受限 (compute-bound)** 区域。

这里讨论的是 **Roofline** 模型中的限制关系，实际执行性能可以位于上限之下。

沿斜线区域向右移动，表示每字节数据参与更多运算，带宽所能支撑的 **FLOP/s** 随之提高；到达屋脊后，继续提高算术强度也不会抬高计算单元本身的吞吐上限。

表 4 给定 GPU 峰值参数与屋脊点。单位分别为 GFLOP/s、GB/s 和 FLOP/B。

GPU	微架构	C_{peak}	B_{peak}	I_*
K40	Kepler	5,040	288	17.5
M40	Maxwell	6,844	288	23.8
P100	Pascal	9,340	732	12.8
V100	Volta	14,028	900	15.6
A100	Ampere	19,500	1,555	12.5
H100	Hopper	66,900	3,352	20.0

同一个 I 在不同硬件上，需要与不同的 I_* 比较。峰值参数是给定计算条件下的性能上限，不能直接用作任意 **kernel** 的预期实测速率。

以 H100 的示例参数为例，采用近似值计算，有

$$I_* \approx \frac{67 \text{ TFLOP/s}}{3.35 \text{ TB/s}} \approx 20 \text{ FLOP/B.}$$

基本矩阵乘法的请求级估计仅约 0.25 FLOP/B，在不计缓存复用的带宽模型中远低于这个屋脊点。

4.3 向量加法与矩阵乘法的对照

4.3.1 向量加法：每个输出都要读二写一

设向量长度为 n ，每个元素为 4 B 的单精度浮点数，计算为

```
z[i] = x[i] + y[i];
```

每个输出需要读取两个输入元素、写入一个输出元素，共 $4 + 4 + 4 = 12 \text{ B}$ ，只进行一次浮点加法。因而

$$F = n, \quad Q = 12n, \quad I_{\text{vec}} = \frac{1}{12} \approx 0.083 \text{ FLOP/B.} \quad (13)$$

这里输出写入占每个元素数据流量的三分之一，必须计入。矩阵乘法每个线程做 W 次乘加而只写一次输出，向量加法则每做一次加法就写一次输出，两者写回成本在总流量中的比重不同。

对 $n = 10^9$ ，运算量为 1 GFLOP，数据量为 12 GB。使用 H100 示例中的 3.35 TB/s 带宽，以及舍入后的 $I = 0.083$ FLOP/B，可得理论极限时间

$$t_{\min} \approx \frac{1 \times 10^9 \text{ FLOP}}{(0.083 \text{ FLOP/B})(3.35 \times 10^{12} \text{ B/s})} \approx 3.60 \text{ ms}. \quad (14)$$

直接用未舍入的数据量计算 $12 \times 10^9 / (3.35 \times 10^{12})$ ，得到约 3.58 ms；差别来自将 1/12 提前舍入为 0.083。

理论极限性能 (*speed of light*) 提供比较实测结果的参照。若测得 kernel 时间 t ，则该模型下的平均有效带宽为 Q/t ，与带宽上限的比值为

$$\eta_B = \frac{Q/t}{B_{\text{peak}}} = \frac{Q/B_{\text{peak}}}{t}.$$

对这一内存受限例子，它等于带宽时间下限与实测时间之比。比较时应沿用相同的数据量、存储层和计时范围。

4.3.2 矩阵乘法：理想复用下的算术强度

矩阵乘法包含 W^2 个点积，每个点积执行 W 次乘法与 W 次加法，因此

$$F = 2W^3.$$

在**理想复用**假设下，两个输入矩阵在所考察存储层各读入一次，输出矩阵写回一次。每个矩阵有 W^2 个 4 B 元素，于是

$$Q_{\text{ideal}} = 3 \times W^2 \times 4 = 12W^2 \text{ B},$$

$$I_{\text{ideal}} = \frac{2W^3}{12W^2} = \frac{W}{6} \approx 0.167W \text{ FLOP/B}. \quad (15)$$

计算量按 W^3 增长，而理想数据量按 W^2 增长，因此算法具有随规模增大的复用潜力。以 $I_* \approx 20$ FLOP/B 为例，理想强度与屋脊相交的条件为 $W/6 \approx 20$ ，即 $W \approx 120$ 。这个交点属于理想复用模型，实际 kernel 的算术强度仍由实现形成的数据流量决定。

4.3.3 相同算法为何得到不同的强度

表 5 矩阵乘法的算法级复用潜力与基本实现的请求级计数。

分析口径	计数的访存行为	算术强度
理想复用	两个输入各读一次，输出写一次	$W/6$ FLOP/B
基本 kernel	每次乘加重新请求两个输入元素	约 0.25 FLOP/B
计入缓存复用	按实际到达所建模层的字节量计数	F/Q ，取决于复用

固定 M_{ik} ，它会参与同一输出行的所有 P_{ij} ，即被不同的 j 使用；固定 N_{kj} ，它会参与同一输出列的所有 P_{ij} ，即被不同的 i 使用。因此，一个输入元素在数学上可以服务多个输出点积。

基本 kernel 将这些点积分给不同线程，但仍在每个线程内部独立请求操作数。缓存可以自然保留部分数据，显式组织复用则进一步利用上述行列关系。共享内存分块的方向是把一组输入先搬入块内可共享的存储，再让多个线程使用这组数据完成各自的计算。

两类例子指向的性能结论

向量加法的强度约为 0.083 FLOP/B，基本矩阵乘法的请求级估计约为 0.25 FLOP/B，都远低于 H100 示例的约 20 FLOP/B 屋脊点。矩阵乘法额外具有显著的输入复用潜力；实现这种复用，可以减小所建模内存层的数据流量，使算术强度向屋脊点移动。

5 例题解析与知识归纳

5.1 三道例题的条件与推导

5.1.1 256 行、4 列矩阵需要多少个线程块

题目。输入矩阵有 256 行、4 列，线程块为 16×16 ，每个线程处理一个输入元素。需要启动多少个线程块？

在忽略原矩阵二维几何形状、通过展平或重映射 (*flattening / remapping*) 自行定义索引的假设下，需要 4 个块。其数量计算为

$$\text{元素数} = 256 \times 4 = 1024, \quad \text{每块线程数} = 16 \times 16 = 256,$$

$$N_{\text{blocks}} = 1024/256 = 4.$$

一个实现这种假设的映射为：将 4 个块沿网格的 x 方向排布，块内仍使用 16×16 个线程。先将块内二维线程索引展平，再映射回原矩阵：

$$\begin{aligned}\ell &= t_y \times 16 + t_x, & 0 \leq \ell < 256, \\ g &= b_x \times 256 + \ell, & 0 \leq g < 1024, \\ \text{row} &= \lfloor g/4 \rfloor, & \text{col} = g \bmod 4.\end{aligned}$$

对应的索引计算片段为

代码 4 由原题的展平假设展开得到的索引片段。

```
1 int localIndex = threadIdx.y * blockDim.x + threadIdx.x;
2 int threadsPerBlock = blockDim.x * blockDim.y;
3 int index = blockIdx.x * threadsPerBlock + localIndex;
4
5 int row = index / 4;
6 int col = index % 4;
```

二维几何覆盖的对照。若继续使用矩阵乘法中 x 对应列、 y 对应行的直接二维映射，则

$$G_x = \left\lceil \frac{4}{16} \right\rceil = 1, \quad G_y = \left\lceil \frac{256}{16} \right\rceil = 16,$$

需要 $1 \times 16 = 16$ 个块。每块只有 4 列线程对应有效元素，其余 12 列需由边界判断跳过。这样共启动 4096 个线程，其中 1024 个有效。

块数随索引方案变化

展平重映射下，4 个块可让全部 1024 个线程各处理一个有效元素。直接二维覆盖下，需要 16 个块覆盖 256 行，并产生无效边缘线程。判断块数时，需要先确定线程如何对应数据坐标。

5.1.2 同一线程块中的线程如何交换数据

题目。某些线程需要读取同一个线程块中其他线程写入的数据。在寄存器、共享内存和全局内存中，哪一种最适合且访问最快？

答案：共享内存。所需存储必须同时满足“同一个块内的线程能够访问同一份数据”和“具有较低访问代价”。共享内存的作用域为线程块，典型位置为 **SM**，正好匹配这两项需求。

寄存器用于每个线程的私有工作值，不能作为这道题中的普通块内共享存储；全局内存支持跨线程访问，但在给定的存储层次中具有更高的访问代价。因此，在三个选项中，共享

内存最合适。

5.1.3 两个线程块写同一地址会得到什么结果

题目。设备端分配一个 `char` 元素，初始值为 3。执行下列 `kernel` 后，`*dst` 的可能值是什么？

代码 5 两个线程块各用一个线程写入同一全局地址。

```
1 __global__ void kernel(char *dst)
2 {
3     dst[0] = blockIdx.x;
4 }
5
6 // dst[0] was initialized to 3 on the device.
7 kernel<<<2, 1>>>(dst);
```

按独立写入先后解释的结果：**0 或 1**。启动配置包含两个线程块，每个块只有一个线程。块 0 向 `dst[0]` 写入 0，块 1 向同一地址写入 1；二者都覆盖原来的值 3。由于块的执行顺序没有保证，在这一写入顺序模型中，最终写入者可以不同。

多个线程块在缺少确定次序的情况下写同一个位置，形成**数据竞争 (data race)**。全局内存的设备可见性允许线程访问该地址，访问权限本身没有确定谁先写、谁后写。

基本矩阵乘法的输出映射让每个有效位置只有一个写入线程，因此其输出写入不具有这道题的同址竞争。判断并行代码时，需要同时检查输入可以共享读取，以及输出是否被多个线程重复写入。

5.2 关键公式与术语

5.2.1 从地址映射到性能上限

下表集中给出前述推导中的关系。 W 为方阵宽度， T 为方形线程块宽度； F 为浮点运算量， Q 为所建模存储层的传输字节量。

问题	关系与条件
二维行主序寻址	$(r, c) \mapsto rW + c$ 。
线程负责的位置	$\text{Row} = b_y B_y + t_y$; $\text{Col} = b_x B_x + t_x$ 。
方阵网格大小	$G_x = G_y = \lceil W/T \rceil$; 总块数为 $G_x G_y$ 。
基本矩阵乘法计数	$F = 2W^3$; $Q_{\text{req}} = 8W^3 + 4W^2 B$ 。
算术强度	$I = F/Q$, 单位为 FLOP/B。
基本矩阵乘法强度	$I_{\text{req}} = W/(4W + 2) \approx 0.25 \text{ FLOP/B}$ 。
向量加法强度	读二写一, $I = 1/12 \approx 0.083 \text{ FLOP/B}$ 。
矩阵乘法理想强度	输入各读一次、输出写一次, $I_{\text{ideal}} = W/6$ 。
Roofline 吞吐上限	$\mathcal{P} \leq \min(C_{\text{peak}}, B_{\text{peak}} I)$ 。
屋脊点	$I_* = C_{\text{peak}}/B_{\text{peak}}$ 。
执行时间下限	$t_{\text{min}} = \max(F/C_{\text{peak}}, Q/B_{\text{peak}})$ 。

存储空间决定数据由哪些线程访问、典型位置在哪里；索引映射决定线程实际读写哪些元素；访存量和算术强度把实现与硬件上限连接起来。分析一个 **kernel** 时，这三个层次应使用相互一致的数据与计数口径。

5.2.2 中英文术语对照

中文	英文	对应含义
寄存器 / 寄存器堆	Register / register file	线程当前工作值所在的低延迟存储。
流式多处理器	Streaming multiprocessor (SM)	包含运算核心及片上存储资源。
共享内存	Shared memory	同一线程块共同访问的存储。
全局内存	Global memory	保存设备可见的大型输入输出。
常量内存	Constant memory	设备线程只读，适于一致访问。
局部内存	Local memory	线程私有，由设备内存支持。
缓存	Cache	保留数据副本以支持后续访问。
作用域 / 生命周期	Scope / lifetime	谁能访问，以及存储存在多久。
行主序	Row-major order	同一行的元素连续存储。
分块	Tile / tiling	将矩阵划分为线程块负责的区域。
归约	Reduction	将多个值汇合为一个结果，例如求和。
展平 / 重映射	Flattening / remapping	在数据与线程坐标之间重新建立对应。

中文	英文	对应含义
浮点运算 / 吞吐率	FLOP / FLOPS rate	运算次数 / 每秒运算次数。
内存带宽	Memory bandwidth	每秒可传输的数据字节数。
算术强度	Arithmetic intensity	每传输一字节对应的运算量。
屋脊点	Ridge point	计算上限与带宽上限的交点。
内存受限 / 计算受限	Memory-bound / compute-bound	由数据供给 / 计算供给主导的限制。
理论极限性能	Speed of light	给定模型下最好可能达到的性能。
数据竞争	Data race	原题中多个线程块无确定次序地写同一地址。